CLASH OF MINDS

AI-Powered Live Debate Evaluation and Evidence Verification System

Complete Project Workflow, NLP Architecture, Real-World Failure Handling, and Exhibition Plan

Prepared for CSES Machine Learning Exhibition
NIT Warangal

Project basis: the provided Clash of Minds architecture proposal, expanded into a practical end-to-end implementation plan.

1. Project Goal

Clash of Minds is an interactive debate platform in which two participants argue opposing sides of a topic and an AI judge evaluates the quality of their arguments. The system supports both keyboard input and voice input. The voice path converts speech into text and then feeds the same textual argument pipeline, so the judging logic remains consistent across both input modes.

The main exhibition mode is a controlled multi-turn debate. Participants alternate turns for a fixed number of rounds. After each turn, the system analyzes the new argument in relation to the topic, the participant's own previous claims, and the opponent's relevant previous claims. Factual claims are separately sent through an Internet-based retrieval and verification pipeline.

The design principle is claim-level evaluation. An entire paragraph is never reduced to one similarity number. The system first identifies the individual claims inside an argument, determines which claims are factual or argumentative, retrieves the relevant debate context, fact-checks checkable claims, and then combines the resulting signals into a structured score.

2. What the Exhibition System Must Handle

1. Two participants with fixed opposing sides, using a common topic and equal turn structure.
2. Both typed and spoken arguments, with transcript confirmation before the NLP judge evaluates a spoken response.
3. Multiple turns in which each participant is expected to listen to and respond to the opponent rather than simply repeat an opening statement.
4. Arguments with different wording but the same meaning, arguments with similar wording but opposite meaning, and arguments that refer back to earlier statements using pronouns or phrases such as "that point" or "your previous claim".
5. Arguments containing one claim, several claims, mixed facts and opinions, conditional claims, rhetorical questions, statistics, and claims requiring more than one source.
6. Internet-based fact verification for checkable factual claims using reliable sources rather than a local-only knowledge base.
7. Continuous operation for approximately 5-6 hours, with repeated match resets and without requiring a model reload between matches.
8. Graceful behavior when speech recognition, internet retrieval, model inference, or browser interaction fails.

3. Core Architecture Principle

The three original NLP pipelines remain the core retrieval and understanding layer: Semantic Embedding Model, Claim Segmentation, and Relevant Point Retrieval. Two additional layers are added around them because the original three are not sufficient to decide whether an argument contradicts an opponent, whether a factual claim is supported by evidence, or whether a rebuttal is actually strong.

Pipeline A - Semantic Embedding Model: creates semantic representations for topic relevance, claim similarity, repetition detection, and fast candidate retrieval.

Pipeline B - Claim Segmentation: breaks long responses into atomic or near-atomic claims and classifies the type of each claim before deeper evaluation.

Pipeline C - Relevant Point Retrieval: retrieves the opponent's most relevant previous claims and also tracks unanswered or partially answered points so the system does not only compare the current response with the immediately previous sentence.

Additional layer - NLI / Argument Relation: uses a cross-encoder to examine the relationship between a current claim and a retrieved opponent claim. Similarity alone cannot distinguish support, contradiction, qualification, or a merely related topic.

Additional layer - Internet RAG Fact Verification: retrieves evidence from reliable internet sources, ranks the evidence, and compares the evidence with the participant's factual claim. RAG retrieval is not treated as proof by itself.

4. Complete User Journey

1. Start match: the operator or exhibition interface creates a new match and clears previous participant state.
2. Enter names: Player 1 and Player 2 are registered.
3. Assign topic and side: the system displays one clearly worded proposition and assigns For and Against.
4. Select input method: each turn can use keyboard text or voice.
5. Submit argument: voice is converted to text, shown to the participant, and optionally corrected before submission.
6. Validate input: the system checks length, emptiness, duplicate submission, active player, and turn state.
7. Segment claims: the response is split into claims or argument units and each unit receives a type label.
8. Run the three core pipelines: semantic embedding, claim segmentation, and relevant point retrieval operate on the current response and debate history.
9. Run relation analysis: the system compares the current response to relevant opponent claims and determines whether it supports, challenges, qualifies, elaborates, or fails to address them.
10. Run fact verification: checkable claims are sent to the Internet RAG path for evidence retrieval and verification.
11. Compute turn score: relevance, reasoning, rebuttal, evidence, clarity, stance consistency, and repetition signals are combined under a fixed rubric.
12. Show structured feedback: the participant sees concise feedback and the current score without exposing internal implementation details.
13. Update debate memory: new claims, response relations, verified evidence, unanswered points, and scores are stored for the next turn.
14. After the final round, aggregate turn-level results, produce the final scorecard and leaderboard entry, and then reset match state for the next pair.

5. Input Layer: Text and Voice

5.1 Voice input and transcript integrity

Problem: Speech-to-text can change or omit important words, especially negation such as "not", "never", or "cannot".

What we do: Convert the audio to text, display the transcript to the participant, allow correction, and only judge the confirmed transcript. Never silently score a transcript that the participant cannot inspect.

What we use: Speech-to-text engine; browser microphone APIs or a tested local/cloud ASR implementation; transcript editor; validation rules.

Expected result: The same clean text format enters the NLP pipeline regardless of whether the original input was typed or spoken.

5.2 Accent and pronunciation differences

Problem: Different accents and speaking styles can produce unequal transcription quality. The debate judge should not penalize an accent simply because the ASR engine misheard it.

What we do: Keep speech quality separate from argument quality. Measure only the transcript after confirmation. If transcription confidence is poor, request a retry or allow text input.

What we use: ASR confidence if available; transcript confirmation; text fallback.

Expected result: The participant is judged on argument content rather than accent.

5.3 Background noise and microphone failure

Problem: An exhibition hall may contain crowd noise, fans, other conversations, or microphone permission problems.

What we do: Provide push-to-talk, visible recording state, maximum recording duration, microphone error handling, and an immediate switch-to-text option.

What we use: Browser microphone status; ASR; timeout; text fallback.

Expected result: A microphone failure cannot terminate the match.

5.4 Mixed language or Hinglish

Problem: Participants may naturally mix languages, which can reduce ASR and NLP reliability if the models are designed primarily for English.

What we do: For the first exhibition version, define English as the supported judging language. Treat multilingual support as an optional extension only after dedicated testing.

What we use: English-focused ASR/NLP stack initially; optional multilingual ASR for a later version.

Expected result: Controlled input improves reliability without pretending multilingual accuracy that has not been measured.

6. Claim Segmentation and Claim Typing

Claim segmentation is central to robustness. The same participant response can contain several independent factual assertions, an opinion, a conditional statement, and a rhetorical question. Each should be handled differently rather than sending the entire paragraph through one verifier.

6.1 Multiple claims inside one response

Problem: A response can contain 2, 3, 4, or many claims. One overall score may hide the fact that some claims are supported while others are unsupported or contradicted.

What we do: Split the response into atomic or near-atomic claims and evaluate each claim independently. Aggregate claim-level results only after each claim has been analyzed.

What we use: Claim segmentation model; sentence splitting; semantic clustering; claim confidence score.

Expected result: Every claim becomes an addressable unit in debate memory and fact verification.

6.2 Mixed fact and opinion

Problem: Statements such as "AI is the future" and "AI increased test scores by 25%" are not the same type of content.

What we do: Classify each segment as factual claim, opinion, prediction, conditional, value judgment, rhetorical/argumentative, statistic, or unclear. Only checkable factual claims should normally trigger web verification.

What we use: Claim-type classifier; lightweight linguistic rules; confidence threshold.

Expected result: The fact checker spends time on claims where verification is meaningful.

6.3 Hidden argument inside a rhetorical question

Problem: A rhetorical question may imply a claim without literally stating it.

What we do: Mark the form as rhetorical/argumentative and use the surrounding debate context to determine whether it expresses an argument. Do not automatically fact-check the literal question as a factual statement.

What we use: Claim-type classifier; debate context.

Expected result: Rhetorical language is evaluated for debate relevance and reasoning instead of producing a false fact-check.

6.4 Conditional and qualified statements

Problem: "AI reduces costs" and "If AI becomes cheaper, it could reduce costs" have different truth conditions.

What we do: Preserve conditions, qualifiers, scope, time, and uncertainty when creating the claim representation. Do not strip terms such as if, only, may, could, not, or unless.

What we use: Transformer-based claim representation; linguistic parsing; claim metadata.

Expected result: The fact checker verifies the actual proposition rather than a simplified but incorrect version.

6.5 Numerical and statistical claims

Problem: A statement containing a number can look authoritative even when the number is incorrect, out of date, or taken out of context.

What we do: Extract value, metric, direction, population, time, condition, and source when available. Search for evidence that matches the exact statistic, not merely the topic.

What we use: Claim parser; number/entity extraction; Internet RAG; evidence verification.

Expected result: The system can distinguish a specific supported statistic from a broad claim that is not established by the source.

7. Semantic Embedding Pipeline

The semantic embedding model is used for meaning-aware retrieval. It should not be treated as the final contradiction or truth detector. The model converts the topic, claims, and previous debate points into vectors. Similarity is then used to retrieve semantically related candidates even when vocabulary changes.

7.1 Different wording, same meaning

Problem: Two arguments may use different vocabulary but express the same idea.

What we do: Use sentence embeddings for semantic retrieval instead of lexical overlap. Validate the embedding model on a custom paraphrase test set before choosing operational thresholds.

What we use: Sentence Transformer such as all-mpnet-base-v2 or a model selected during benchmarking; cosine similarity only for retrieval/ranking.

Expected result: Semantically related points can be retrieved even without shared keywords.

7.2 Similar wording, opposite meaning

Problem: Adding or changing a negation can reverse the meaning while keeping most words identical.

What we do: Do not infer agreement or contradiction from embedding similarity. Use a cross-encoder/NLI model and a stance signal after retrieval.

What we use: Sentence embeddings for retrieval; DeBERTa-family NLI/cross-encoder or equivalent for relation analysis; explicit negation-sensitive test cases.

Expected result: Similarity answers "related?" while NLI/stance answers "what relation?".

7.3 Context changes meaning

Problem: A phrase can only make sense because it refers to an earlier statement in the debate.

What we do: Embed and retrieve claim-level context, not isolated turns. The current claim is evaluated together with the relevant retrieved history.

What we use: Sentence Transformer; debate-memory store; contextual claim representation.

Expected result: Pronouns and contextual references have access to the necessary prior content.

7.4 Repetition with different vocabulary

Problem: A participant can repeat the same idea using new wording and appear to provide a new argument.

What we do: Compare the new claim with the participant's own previous claim embeddings. Then classify the result as repeated, rephrased, elaborated, or repeated-with-new-evidence rather than imposing a penalty from similarity alone.

What we use: Sentence embeddings; claim history; similarity thresholds tuned on examples; optional claim-relation classifier.

Expected result: Repetition penalties apply to genuine repeated reasoning, not legitimate elaboration.

8. Relevant Point Retrieval and Debate Memory

Relevant Point Retrieval is the mechanism that makes the debate genuinely interactive. The system should not assume that the current participant is responding to the immediately previous sentence. It should retrieve the opponent points that are most relevant and, where appropriate, unresolved.

8.1 Does the response address the opponent at all?

Problem: A participant may speak about the general topic without responding to the opponent's actual point.

What we do: Retrieve candidate opponent claims from the debate history, rerank them with a cross-encoder, and measure whether the current response actually addresses one or more of those claims.

What we use: Embedding retrieval followed by cross-encoder reranking; debate-history store; argument-relation classifier.

Expected result: The system can separate "same topic" from "directly addresses opponent point".

8.2 The participant repeats an old point instead of responding to the latest point

Problem: Simply comparing the new response with the latest opponent turn can cause the system to miss an older but still unanswered claim, or reward repeated old arguments.

What we do: Maintain explicit claim status: unanswered, partially answered, addressed, or resolved. Retrieval ranks unresolved and relevant claims while retaining full history for context.

What we use: Debate-memory structure; claim status tracking; embeddings; recency and unresolved-point features.

Expected result: The response is evaluated against the right debate history, not just the last sentence.

8.3 Multiple opponent points can be addressed at once

Problem: One response may challenge two, three, or more previous claims.

What we do: Retrieve top-k candidate opponent claims, evaluate each candidate pair, and keep all materially relevant relationships instead of forcing a single matched point.

What we use: Top-k vector retrieval; cross-encoder reranking; relation classifier.

Expected result: A response can receive separate engagement signals for multiple opponent claims.

8.4 Pronouns and references such as "that", "this", or "your previous point"

Problem: The current sentence may be incomplete without the previous turns.

What we do: Use the recent turn window plus retrieved older claims to reconstruct the relevant context. Preserve claim references in debate memory.

What we use: Recent context window; claim embeddings; relevant-point retrieval; optional coreference-aware preprocessing.

Expected result: The system can associate references with the appropriate earlier claim.

8.5 Logical relation is more nuanced than contradiction

Problem: A rebuttal may partially agree, qualify a point, introduce a limitation, or elaborate without directly contradicting it.

What we do: Use relation labels such as support, contradict, qualify, elaborate, neutral, and unrelated. Then derive rebuttal quality from the relation plus reasoning and evidence signals.

What we use: Cross-encoder/NLI model; debate-specific relation classifier or calibrated relation rules.

Expected result: The judge distinguishes disagreement from meaningful rebuttal.

9. Internet RAG Fact-Verification Pipeline

The fact-checking layer is Internet-based rather than a local-only RAG database. Its job is to retrieve evidence for checkable claims and determine whether the retrieved evidence supports, refutes, partially supports, or fails to establish the claim. Search retrieval and verification are separate steps.

Sources should be prioritized by reliability and relevance. Suitable sources include government sites, official institutions, research papers, reputable reference sources such as Wikipedia for general background, and established news organizations when the claim is time-sensitive. No single source type should be treated as universally correct for every topic.

9.1 Search result is relevant but is not evidence

Problem: A search result can mention a topic without actually supporting the specific claim.

What we do: Retrieve passages, not just URLs. Re-rank passages against the claim and only use the passages that directly bear on the claim.

What we use: Internet search/retrieval; passage extraction; cross-encoder reranker; source metadata.

Expected result: The verifier sees claim-relevant evidence rather than a generic topic page.

9.2 Sources disagree

Problem: Two reliable sources can report different numbers, dates, interpretations, or findings.

What we do: Retrieve multiple sources, retain conflicting evidence, and return a conflict/uncertainty status instead of blindly selecting the first result.

What we use: Multi-source retrieval; source metadata; evidence comparison; verification model.

Expected result: The system can report conflicting evidence rather than pretending there is one certain answer.

9.3 Source quality varies

Problem: A random blog and a primary research report should not automatically receive equal evidential weight.

What we do: Use source-quality rules and metadata to prioritize primary or authoritative sources. A lower-quality source may be used as a lead but not as the sole basis for a strong verification verdict.

What we use: Source-domain allowlist/priority rules; source-type classifier; metadata scoring.

Expected result: Evidence quality becomes part of the verification result.

9.4 Outdated information

Problem: A claim can depend on the date, especially for technology, policy, population figures, prices, or current events.

What we do: Record source publication date and retrieval date. Prefer recent authoritative sources when the claim is time-sensitive.

What we use: Date extraction; source metadata; query generation using time constraints when required.

Expected result: Time-sensitive claims are not judged using stale evidence when newer authoritative evidence exists.

9.5 No reliable evidence found

Problem: Failure to find evidence does not prove that the participant is wrong.

What we do: Return insufficient evidence rather than refuted. Keep the evidence-quality score conservative and continue evaluating the rest of the debate.

What we use: Retrieval confidence thresholds; verifier confidence; explicit insufficient-evidence state.

Expected result: No false certainty is introduced by a failed search.

9.6 Partial support

Problem: A source may support only one part of a multi-part claim.

What we do: Verify each atomic claim separately and aggregate them. Do not label the entire sentence as fully supported when only one component is supported.

What we use: Claim segmentation; per-claim retrieval; per-claim verification; aggregation logic.

Expected result: Multi-claim arguments receive claim-level evidence outcomes.

10. Fact Verification Decision Logic

The recommended output states are SUPPORTED, PARTIALLY SUPPORTED, REFUTED, INSUFFICIENT EVIDENCE, and AMBIGUOUS. The verifier should also return evidence excerpts, source information, and a confidence or uncertainty indicator.

1. Extract the factual claim exactly as expressed by the participant, preserving qualifiers and numbers.
2. Generate a focused search query from the claim. For numerical claims, include the relevant entities, metric, and time context where available.
3. Retrieve multiple candidate sources from the Internet.
4. Extract relevant passages from those sources.
5. Rank the passages against the claim using semantic retrieval followed by a more precise cross-encoder reranker.
6. Run claim-evidence verification using an NLI/verification model and source-quality signals.
7. Check whether the evidence supports the entire claim, only part of it, or contradicts it.
8. Return the verdict with evidence and uncertainty instead of forcing a binary true/false decision.

11. Debate Scoring Layer

Scoring is a deterministic aggregation layer over several independently measured signals. The system should not use a single model to predict a winner directly. This makes the judge easier to test, explain, and debug.

Topic relevance - 15%: semantic relation between the topic/question and the participant's actual claims.

Clarity - 15%: structure, readability, and coherent expression of the argument. This should not reward unnecessarily complex vocabulary.

Logical reasoning - 20%: quality of the reasoning connecting claims to the participant's position, supported by argument structure signals.

Counterargument / rebuttal - 20%: whether the participant directly addresses relevant opponent claims and provides a meaningful response.

Evidence quality - 20%: quality and support of factual claims, with Internet RAG evidence and uncertainty handling.

Stance consistency - 10%: whether the participant remains aligned with the assigned side while allowing nuanced qualifications.

These weights are an initial engineering rubric. They should be calibrated and tested on manually labeled examples before the exhibition rather than treated as mathematically optimal by default.

12. Important Fairness and Interpretation Rules

1. A contradiction is not automatically a strong rebuttal. "That is wrong" contains disagreement but little reasoning.
2. A true fact is not automatically a strong argument. Evidence quality and logical reasoning remain separate dimensions.
3. A participant should not be rewarded simply for using complicated English. Clarity and reasoning are scored separately from vocabulary sophistication.
4. A repeated claim should not always be penalized if the participant adds new evidence or materially stronger reasoning.

5. A factual verification failure should not automatically decide the entire match. It primarily affects the evidence component unless it materially changes the argument itself.
6. Unverified claims should be labeled unverified or insufficiently supported instead of being presented as definitively false.

13. Live Multi-Turn Debate Workflow

1. Round 1: Player 1 opening argument. The system evaluates topic relevance, claims, stance, reasoning, and evidence.
2. Round 1 response: Player 2 opening argument. The system starts building the opponent-claim graph and unresolved-point list.
3. Round 2: Each participant responds after seeing the opponent's previous contribution. Relevant-point retrieval becomes active.
4. Round 3: Participants continue with counterarguments, evidence, qualifications, and direct rebuttals. The system checks repetition and unresolved claims.
5. Final round: Each participant gives a closing argument. The system evaluates the final response using the accumulated debate memory.
6. Final score: Turn scores are aggregated with bounded penalties for repetition or unsupported claims and without allowing one anomalous turn to dominate the entire match.

The main exhibition configuration should use a fixed number of rounds and a fixed speaking/submission limit. This keeps the experience predictable, keeps both participants under the same conditions, and prevents an extremely long response from dominating simply because it contains more text.

14. Latency Strategy

Low latency comes from architectural separation rather than removing the deeper NLP components. The fast path handles operations that are required for immediate debate progression, while the fact-checking path can run in parallel where practical.

1. Preload the local NLP models once when the application starts. Do not reload models for every participant or every turn.
2. Use semantic embeddings to retrieve a small candidate set rather than sending the entire debate history into an expensive cross-encoder.
3. Use cross-encoder/NLI only on top-k relevant candidate pairs.
4. Run factual-claim retrieval in parallel with debate-relation analysis when the backend architecture permits it.
5. Cache repeated fact-check queries and repeated retrieval results during the exhibition.
6. Bound argument length and maximum claim count so inference cost cannot grow without limit.
7. Use explicit timeouts for Internet operations. A slow web search should not freeze the entire debate UI.
8. If RAG is delayed, present the debate score with the evidence component marked pending/unverified, then update the evidence result when the verification completes or when the timeout occurs.

15. Failure Handling and Exhibition Reliability

15.1 Internet unavailable

Problem: The RAG layer cannot retrieve new evidence.

What we do: Keep the debate NLP engine operational. Mark fact verification as temporarily unavailable and do not invent a verdict.

What we use: HTTP timeout; retry policy; pending/unverified status; cached results for repeated claims.

Expected result: A web outage affects evidence verification, not the entire match.

15.2 Model inference failure

Problem: A model can fail because of malformed input, resource exhaustion, or an unexpected runtime error.

What we do: Catch exceptions at the pipeline boundary, log the failing stage, and return a controlled fallback status. The UI should remain responsive and allow a retry.

What we use: Python exception handling; structured logging; bounded input; watchdog/health checks.

Expected result: A single bad argument does not crash the application.

15.3 Duplicate or out-of-order submissions

Problem: Fast clicks, browser retries, or multiple tabs can corrupt turn order.

What we do: Validate player ID, match ID, turn ID, and server-side turn state before accepting a submission. Make processing idempotent for a turn ID.

What we use: FastAPI validation; unique turn IDs; server-side state machine; database or in-memory state store.

Expected result: Only one valid result is committed per turn.

15.4 Browser refresh or participant abandonment

Problem: The page may reload or a participant may leave before finishing.

What we do: Store active match state on the backend, provide an operator reset, and expire abandoned matches after a timeout.

What we use: Backend session state; reset endpoint; inactivity timeout.

Expected result: The system can recover without restarting the application.

15.5 Long-running memory growth

Problem: Repeated matches can accumulate embeddings, transcripts, cache entries, or temporary objects.

What we do: Separate persistent model state from per-match state. Bound caches and histories. Delete match-specific objects on reset and monitor RAM/VRAM during stress testing.

What we use: Explicit match state object; cache size limits; Python garbage collection where needed; GPU cache management if applicable.

Expected result: Memory use stays bounded across many matches.

15.6 Search rate limits

Problem: A five-to-six-hour exhibition can generate many Internet searches.

What we do: Cache repeated claims, combine related retrieval where possible, cap requests per turn, and handle rate-limit responses without crashing the system.

What we use: Caching; request counters; backoff; timeout and error states.

Expected result: The fact-checking subsystem degrades gracefully under API or web limits.

16. Debate Memory Design

The debate memory should be structured rather than a raw transcript. Each participant has a claim history, each opponent claim has a response status, and each claim may have associated evidence and relation labels.

Topic state: topic text, proposition, assigned stance for each player.

Claim state: claim text, player, turn, claim type, embedding, stance, verification status.

Response state: which opponent claims are addressed, partially addressed, or still unanswered.

Evidence state: retrieved sources, evidence passages, verification verdict, source date, and verification confidence.

Scoring state: per-turn criterion values and aggregate score.

On reset, all match-specific state is cleared while model weights and static configuration remain loaded. This is the key distinction between a persistent application process and a temporary debate session.

17. Secondary One-Shot Mode

The system can also support a simpler mode in which both participants submit a longer opening argument and the system evaluates them without live rebuttal. This mode should be treated as a backup or optional demonstration mode, not as the flagship experience, because it does not exercise the full relevant-point retrieval and cross-turn reasoning pipeline.

In one-shot mode, the same claim segmentation, semantic embedding, RAG verification, stance, and scoring components remain usable. Only the live debate memory and response-to-opponent logic become less important.

18. Proposed Technology and Model Stack

The exact model versions should be finalized only after benchmarking on the robustness test set. The following stack is the initial engineering proposal.

Speech-to-text: a tested speech recognition engine; browser Web Speech API can be the first prototype, with a more controllable ASR option evaluated for exhibition deployment.

Semantic embeddings: Sentence Transformers, initially benchmark all-mpnet-base-v2 against a smaller faster alternative if latency requires it.

Cross-encoder / NLI: a DeBERTa-family model trained for NLI or an equivalent cross-encoder, benchmarked on negation, contradiction, qualification, and neutral relations.

Claim segmentation: a transformer-based classifier/sequence labeling approach plus deterministic preprocessing rules for sentence boundaries. Exact model chosen after testing multi-claim examples.

Relevant point retrieval: embedding retrieval followed by top-k cross-encoder reranking, with debate-memory status features.

RAG retrieval: Internet search/retrieval, source filtering, passage extraction, semantic retrieval, and cross-encoder reranking.

Fact verification: NLI/claim-evidence verification model plus source-quality and scope checks.

Backend: FastAPI with asynchronous endpoints and WebSocket updates for the live interface.

Frontend: React/Vite or equivalent web interface with transcript display, timers, turn controls, and score panels.

Storage: SQLite or another lightweight store for match results, with in-memory active debate state.

19. Core Libraries

- FastAPI, Uvicorn, Pydantic, WebSockets / asyncio for backend control and live updates.
- PyTorch and Hugging Face Transformers for neural inference.
- Sentence Transformers and scikit-learn for embeddings, cosine similarity, and evaluation.
- NumPy for vector operations and numerical processing.
- FAISS or another local in-process index only for fast retrieval over the active debate history; it is not the factual knowledge base.
- Requests/httpx and HTML parsing tools for Internet evidence retrieval.
- PyMuPDF where PDF-based web evidence needs text extraction.
- RapidFuzz for lightweight lexical duplicate detection alongside semantic comparison.
- SQLite / sqlite3 for match logs and results.
- Pytest and HTTPX for unit, integration, and API testing.
- Optional ONNX Runtime / Optimum after the baseline pipeline is measured and optimization targets are known.

20. Robustness Test Plan Before Exhibition

The project should not go directly from coding to exhibition. A dedicated robustness dataset should be created from manually written and reviewed examples that represent the situations visitors are likely to produce.

1. Paraphrase cases: different vocabulary with the same meaning.

2. Negation cases: identical wording with a change in polarity.

3. Context-reference cases: pronouns and phrases referring to previous turns.

4. Repeated-claim cases: same idea rewritten, with and without new evidence.

5. Multi-claim cases: 2, 3, 4, and more claims in one response.

6. Fact/opinion cases: mixed checkable and non-checkable statements.

7. Conditional cases: if, may, might, only if, unless, and similar qualifiers.

8. Statistical cases: exact numbers with varying context or source quality.

9. Rhetorical-question cases: argumentive questions that should not be treated as literal facts.

10. Rebuttal cases: directly addressed, partially addressed, unrelated, and old-unanswered opponent points.

11. Multiple-point cases: one response intentionally addresses several previous opponent claims.

12. Evidence cases: supported, refuted, partially supported, conflicting, and insufficient-evidence claims.

13. Voice cases: controlled accent variation, background noise, speech speed changes, and negation-sensitive phrases.

14. System cases: duplicate submission, refresh, reset, timeout, Internet loss, and repeated-match memory growth.

Evaluation should track accuracy/F1 for classifiers and relation labels, retrieval recall for relevant point retrieval, verification precision/recall for fact-checking, word error rate for speech recognition where applicable, and latency/RAM/VRAM across repeated matches. Exact target values should be established after the baseline measurements rather than invented in advance.

21. Final End-to-End Workflow

1. A new match is created and the topic plus opposing sides are fixed.

2. A participant selects text or voice input.

3. If voice is selected, speech-to-text creates a transcript; the participant reviews and confirms or edits it.
4. The backend validates the submission and assigns a unique turn ID.
5. Claim Segmentation identifies each argument unit and classifies whether it is factual, opinion-based, conditional, rhetorical, statistical, or otherwise.
6. The Semantic Embedding Model encodes the topic, current claims, and debate-history claims.
7. Topic relevance and semantic retrieval use the embeddings to find the most relevant historical points without relying on exact vocabulary overlap.
8. Relevant Point Retrieval selects candidate opponent claims and candidate own claims. It considers semantic similarity, recency, and whether a point is still unanswered or partially answered.
9. A cross-encoder/NLI stage deeply compares the current response with the selected opponent claims and classifies relations such as support, contradiction, qualification, elaboration, neutral, or unrelated.
10. The system separately checks stance consistency and semantic repetition against the participant's own history.
11. Every checkable factual claim enters the Internet RAG path: focused search, multi-source retrieval, passage extraction, evidence reranking, claim-evidence verification, and source-quality assessment.
12. The scoring engine combines relevance, clarity, reasoning, rebuttal, evidence, stance consistency, and repetition signals into a turn score.
13. The interface displays concise feedback and the score, while the backend updates debate memory.
14. The next participant turn repeats the process using the updated history.
15. After the final round, the system aggregates the turn scores, presents the final scorecard and leaderboard, and records the result.
16. The operator resets the match. All participant-specific memory, claims, temporary evidence, and scores are cleared while loaded models remain in memory for rapid reuse.

22. What We Will Deliberately NOT Do

- We will not use cosine similarity alone to determine contradiction, agreement, or debate quality.
- We will not treat the first web search result as proof of a factual claim.
- We will not send every long argument directly into one monolithic model and expect a single winner score.
- We will not force every sentence into true/false when evidence is incomplete or the statement is an opinion or prediction.
- We will not penalize participants for accent, vocabulary complexity, or a speech-recognition error that the participant can correct.
- We will not reload the NLP models after every match.
- We will not claim 0 percent error. Instead, we will demonstrate measured robustness on a defined test set and provide safe uncertainty behavior for cases the system cannot confidently resolve.

23. Recommended Build Order

1. Build and benchmark semantic embeddings on paraphrase, relevance, negation, and repetition examples.
2. Build claim segmentation and claim-type classification using manually labeled multi-claim examples.
3. Build relevant point retrieval over a small simulated debate history and validate that the correct opponent points are retrieved.

4. Add cross-encoder/NLI relation analysis and test support, contradiction, qualification, and unrelated examples.
5. Build the Internet RAG fact-checking prototype independently. Validate evidence retrieval before connecting it to scoring.
6. Create the scoring engine and calibrate weights/thresholds against human-labeled examples.
7. Connect the three NLP pipelines, relation analysis, RAG, and debate memory into the backend.
8. Add voice input and transcript confirmation.
9. Build the live frontend and fixed-round debate flow.
10. Perform long-run stress tests with repeated matches, Internet failures, duplicate submissions, resets, and memory monitoring.
11. Only after the baseline passes the robustness tests, add ONNX or other optimization if measured latency justifies it.

24. Exhibition Readiness Checklist

- A clean topic list with clearly worded propositions and fixed For/Against assignments.
- A tested speech-to-text path plus reliable text fallback.
- A visible transcript correction step for spoken input.
- Fixed round count and timers so both participants receive the same interaction conditions.
- Claim-level semantic analysis and debate-memory tracking.
- Relevant-point retrieval that can explain which opponent claim a response addressed.
- Internet RAG with multiple-source retrieval and clear insufficient-evidence handling.
- Fast-path debate scoring that does not freeze when web retrieval is slow.
- Per-match reset and operator recovery controls.
- Logging, health checks, request timeouts, and repeated-match stress-test results.
- A robustness report showing performance on paraphrase, negation, multi-claim, retrieval, fact-checking, voice, and failure cases.

25. Final Project Position

Clash of Minds should be presented as a modular NLP judging system rather than a single "AI judge" model. The strongest part of the architecture is the separation of responsibilities: semantic embeddings handle meaning-aware retrieval; claim segmentation creates atomic units; relevant point retrieval connects the response to the real debate history; NLI/argument-relation analysis determines the relationship between claims; Internet RAG verifies factual assertions; and a deterministic scoring layer combines these signals into a structured judgment.

This architecture is intentionally designed around the real-world failure cases expected at a live exhibition. It keeps the main debate interaction fast, gives the fact-checker access to broad Internet information, prevents individual failures from taking down the entire application, and keeps match-specific memory isolated so the same system can run repeatedly throughout the exhibition.